



Universidade do Minho

Licenciatura em Engenharia Informática - LEI

Computação Gráfica

Trabalho Prático - Fase 4

Grupo 14

A107324 - David José Barbosa Alves

A107368 - Diogo Malheiro Pais

A107377 - João Pedro Ferreira Gomes

A107332 - Pedro Afonso Quesado Pacheco Pereira

1. Introdução

Esta fase representa a conclusão do motor gráfico, completando o *pipeline* de renderização com iluminação e texturização. Partindo de toda a infraestrutura construída nas fases anteriores, geração de primitivas (Fase 1), *Scene Graph* hierárquico (Fase 2), VBOs e animações com curvas de Catmull-Rom (Fase 3), as alterações desta fase foram deliberadamente cirúrgicas: a arquitetura funcional prévia manteve-se intacta.

As duas componentes centrais sofreram as seguintes evoluções:

- **Generator:** Após não ter sofrido alterações nas Fases 2 e 3, o gerador voltou a ter um papel central. O formato de saída .3d e a estrutura de dados interna foram expandidos para incluir o cálculo matemático de normais e coordenadas de textura (UV) por vértice. Os algoritmos de geração das primitivas já existentes foram atualizados para suportar esta expansão e uma nova primitiva (anel) foi adicionada.
- **Engine:** O motor passou a gerir a memória através de três VBOs distintos por modelo (vértices, normais e coordenadas UV). Adicionalmente, foi implementada a gestão de texturas através da biblioteca externa DevIL, a aplicação de propriedades de materiais lidas da configuração XML, e a parametrização de múltiplas fontes de luz.

2. Expansão do Generator: Normais e Coordenadas UV

A estrutura de dados dos vértices foi atualizada para conter os oito campos numéricos necessários ao *pipeline* de iluminação e texturização (três para posição, três para a normal, e dois para as coordenadas de textura). O protocolo de ficheiros .3d refletiu esta alteração, passando a guardar oito valores por linha, mantendo o cabeçalho original com o número total de vértices para alocação dinâmica no motor.

2.1 Esfera

Sendo uma superfície perfeitamente redonda e equidistante do centro, a normal em cada ponto da esfera corresponde simplesmente ao seu próprio vetor posição devidamente normalizado. As coordenadas UV mapeiam diretamente os ângulos paramétricos α e β para o intervalo normalizado, resultando numa projeção equiretangular totalmente compatível com as texturas planetárias utilizadas na cena de demonstração.

2.2 Plano

A normal é constante e aponta verticalmente para cima. As coordenadas UV crescem linearmente com o índice da célula da grelha gerada, cobrindo o espectro da textura de forma perfeitamente distribuída, independentemente do número de divisões parametrizadas.

2.3 Caixa (Box)

Cada face da caixa possui uma normal fixa e ortogonal à sua própria direção. A orientação *Counter-Clockwise* (CCW) dos triângulos, garantida desde a Fase 1, assegura que as normais apontam corretamente para o exterior do sólido geométrico. As coordenadas de textura são aplicadas de forma independente a cada uma das seis faces.

2.4 Cone

A base do cone utiliza uma normal fixa a apontar para baixo. Já a superfície lateral requereu cálculo trigonométrico explícito: a inclinação natural do cone implica que a normal possua uma componente vertical e uma componente horizontal, determinadas pela relação matemática entre o raio e a altura. Para garantir um *Smooth Shading* contínuo da superfície (em concordância com os testes de referência da UC), a normal de cada vértice lateral foi calculada conjugando estas duas componentes, originando gradientes de luz suaves.

2.5 Patches de Bézier

As normais para estas superfícies complexas foram calculadas analiticamente a partir das suas derivadas parciais, aproveitando as derivadas dos polinómios de Bernstein implementados na Fase 3. O produto vetorial dos dois vetores tangentes da superfície origina a normal exata naquele ponto. Foi crucial garantir a ordem correta na execução deste produto vetorial para assegurar que as normais calculadas apontassem para o exterior do modelo (como verificado no Bule de Utah), permitindo a correta incidência da luz. O mapeamento UV seguiu naturalmente os parâmetros bidimensionais da iteração da grelha.

2.6 Nova Primitiva: Anel (Ring)

A simulação dos anéis de Saturno exigiu a criação de uma coroa circular plana, gerada através de um raio interno e externo. A primitiva é dupla, gerando uma face superior (com normal positiva no eixo vertical) e uma face inferior (com normal negativa e ordem de desenho de vértices invertida para respeitar a convenção CCW). A transparência do modelo final é tratada pelo *Engine* através do

blending do OpenGL.

3. Engine: Três VBOs por Modelo

O carregamento de geometria foi reformulado. Durante o *parsing* dos ficheiros .3d, o motor separa os dados lidos em três vetores na memória principal (CPU) e efetua a alocação de três VBOs distintos na placa gráfica (VRAM): um exclusivo para posições, outro para normais e um terceiro para as coordenadas de textura.

Para maximizar o desempenho no processo de renderização, os três *Client State Arrays* correspondentes são ativados globalmente na função principal antes da travessia hierárquica do *Scene Graph*, sendo desativados apenas no final da *frame*. Consequentemente, durante o ciclo recursivo, o motor necessita apenas de associar os identificadores dos VBOs do modelo iterado aos respetivos ponteiros do OpenGL imediatamente antes da chamada de desenho (*draw call*). Toda a estrutura de grupos, o uso da *Matrix Stack* e o cálculo de animações por *splines* mantiveram-se inalterados.

4. Materiais

O sistema de renderização passou a suportar a associação de propriedades físicas aos modelos. O carregador de XML inicializa uma estrutura de Material com valores neutros, que são sobrepostos caso sejam encontradas definições específicas de cor na cena. O motor lê os canais de cor no espectro padrão, normaliza-os, e aplica estas propriedades antes de desenhar o modelo correspondente.

A componente emissiva (emissive) revelou-se particularmente relevante na cena de demonstração: astros como o Sol e o fundo de estrelas têm esta componente definida no seu valor máximo. Isto torna-os completamente autoiluminados e visualmente independentes das fontes de luz da cena, representando de forma otimizada corpos celestes que emitem luz própria sem forçar o motor a calcular emissões globais dispendiosas.

5. Texturização

A texturização foi alcançada através da integração da biblioteca Devil (*Developer's Image Library*), responsável por decodificar formatos de imagem padrão (JPEG/PNG) e extrair os *arrays* de *pixels* para o carregador de texturas bidimensionais do OpenGL. O problema técnico mais comum na importação destas imagens é a inversão do eixo vertical na renderização; a solução implementada

passou por instruir a biblioteca a inverter o eixo vertical nativamente durante a fase de leitura, evitando penalizações de desempenho com remapeamentos posteriores.

No *Scene Graph*, a arquitetura foi desenhada para associar um vetor de identificadores de textura a cada modelo geométrico, o que cria uma infraestrutura flexível, preparada para cenários futuros de *multitexturing*. Na versão atual do *pipeline*, o motor ativa a textura primária do vetor, desativando o estado da máquina de estados do OpenGL em modelos geométricos puros, para evitar erros de oclusão visual com as propriedades dos materiais.

6. Iluminação

O motor suporta a ativação simultânea de até 8 fontes de luz independentes, que são extraídas da configuração XML e posicionadas no espaço a cada iteração da cena antes do processo de desenho.

6.1 Tipos de Luz e Leitura do XML

A arquitetura distingue automaticamente três comportamentos luminosos através do valor do quarto componente homogêneo (w) do vetor de posição, seguindo as convenções matemáticas do OpenGL:

- **Pontual:** Definida com $w=1.0$, atua irradiando luz em todas as direções a partir de uma origem específica.
- **Direcional:** Definida com $w=0.0$, simula uma fonte luminosa a uma distância infinita, onde as coordenadas de posicionamento são rigorosamente interpretadas como o vetor de direção paralela dos raios de luz.
- **Spotlight:** Mantém o comportamento pontual ($w=1.0$), mas sofre restrições de emissão através de um ângulo de corte (*cutoff*) e de um vetor de direção explícito.

6.2 Decisões de Implementação

Foram tomadas decisões técnicas cruciais na fase de inicialização do contexto gráfico para garantir resultados fidedignos, respondendo diretamente aos desafios propostos nas notas de lançamento da Fase 4:

- **GL_RESCALE_NORMAL:** Ativado globalmente na inicialização para garantir que as normais sofrem renormalização após a aplicação de transformações geométricas. Sem isto, a iluminação incidiria de forma incorreta sobre modelos afetados pela *tag* `<scale>`.
- **Luz Ambiente Global (GL_LIGHT_MODEL_AMBIENT):** O modelo foi configurado com um vetor branco (1.0, 1.0, 1.0). Esta decisão permite que as cores ambientes especificadas

nos materiais (GL_AMBIENT) sejam corretamente reproduzidas sem a necessidade imperativa de definir uma componente ambiente individual em todas as luzes da cena.

No *pipeline* de desenho, as posições das luzes são submetidas logo após as transformações de câmara (gluLookAt) e antes da invocação de qualquer renderGrupo. Esta rigorosa ordem de operações assegura que o OpenGL fixa as coordenadas das luzes em espaço de mundo (*World Space*) e não em espaço de objeto, garantindo que estas não rodam ou transladam inadvertidamente juntamente com a geometria local.

7. Cena Demo: Sistema Solar Texturizado

A cena de demonstração final é a evolução direta do sistema solar dinâmico construído na Fase 3. A complexa hierarquia de grupos, os *offsets* angulares (para evitar a sizígia), as órbitas geradas por curvas de Catmull-Rom e as rotações temporizadas foram mantidos intactos. A evolução centrou-se exclusivamente na camada visual: cada corpo celeste passou a ter uma representação fotorealista através de mapeamento de texturas e propriedades de material apropriadas.

Configurações relevantes no sistema_solar.xml:

- **Fundo estelar:** Uma esfera de raio elevado com a textura stars_milky_way.jpg e material de componente emissiva branca no máximo, não sendo afetada pela atenuação da luz da cena.
- **Sol:** Utiliza a textura sun.jpg e um material emissivo total. A principal fonte de luz pontual do sistema está posicionada na origem (0, 0, 0), coincidindo perfeitamente com a malha do Sol.
- **Planetas:** Cada astro foi revestido com texturas cilíndricas reais de alta definição (ex: mercury.jpg, venus.jpg, mars.jpg, jupiter.jpg, saturn.jpg, uranus.jpg, neptune.jpg).
- **Terra:** Configurou-se um caso especial que atesta a flexibilidade da arquitetura, declarando múltiplas texturas na hierarquia do modelo (mapa diurno e noturno), sendo renderizada no *pipeline* atual com o mapa diurno fotorealista.
- **Anel de Saturno:** Recurso à nova primitiva plana circular (ring) mapeada com a textura saturn_ring.png, renderizada com transparências através da ativação rigorosa de GL_BLEND.
- **Cometa (Utah Teapot):** O cometa orbitante que demonstra as patches de Bézier foi mantido da Fase 3 e texturizado com sucesso utilizando o teapot.jpg.

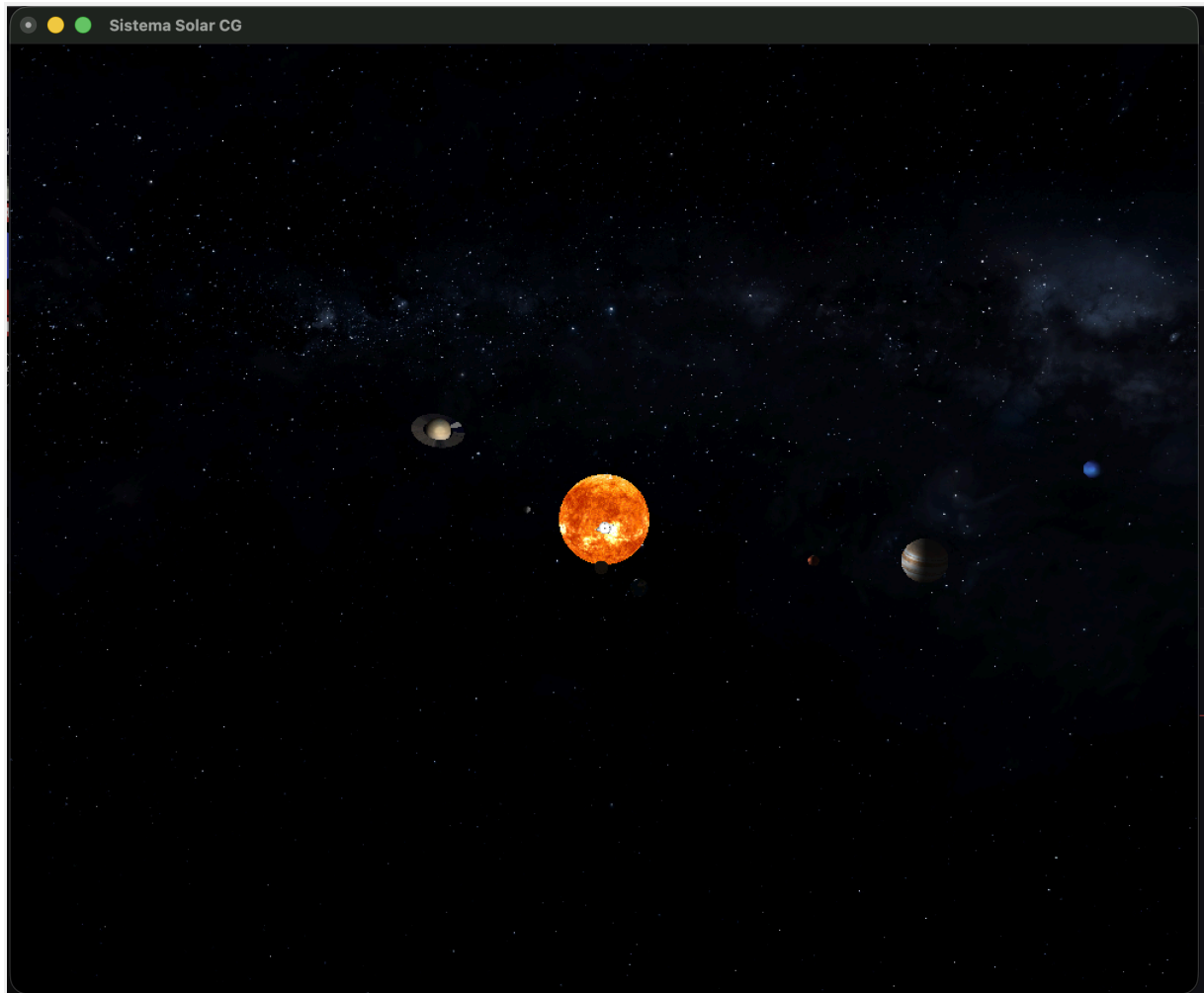
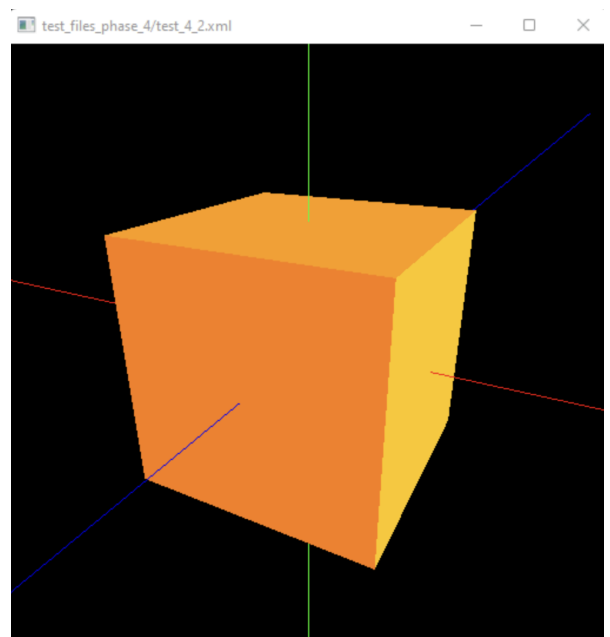
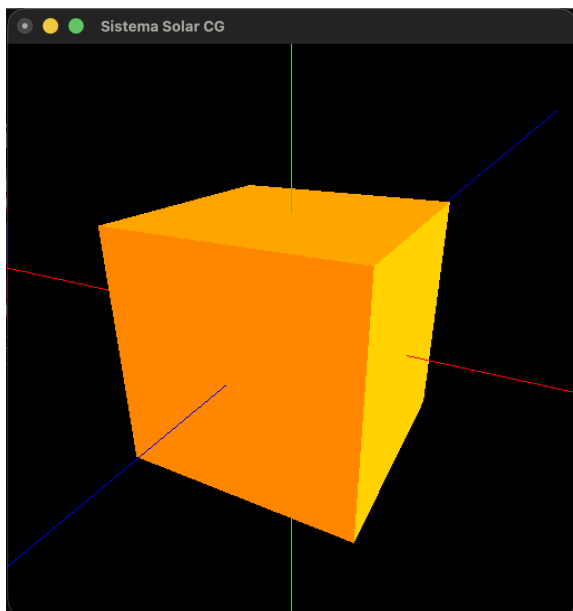
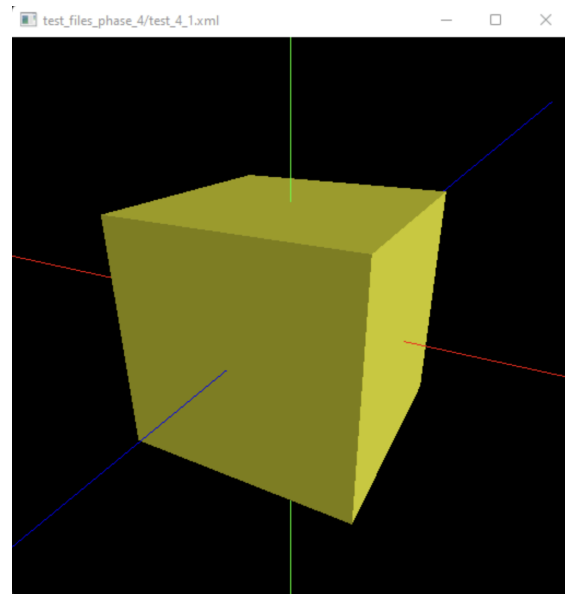
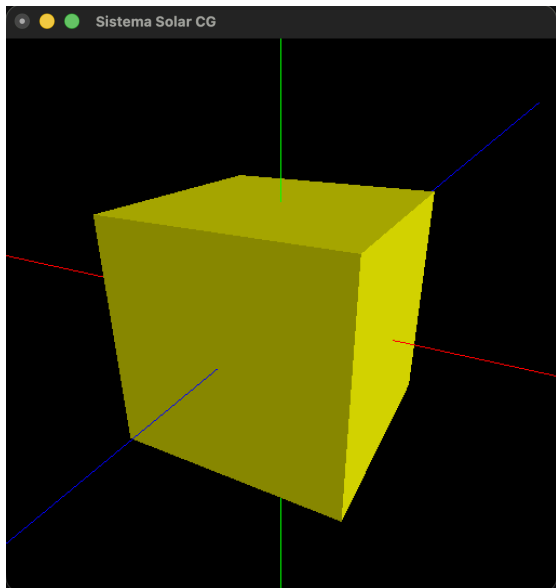


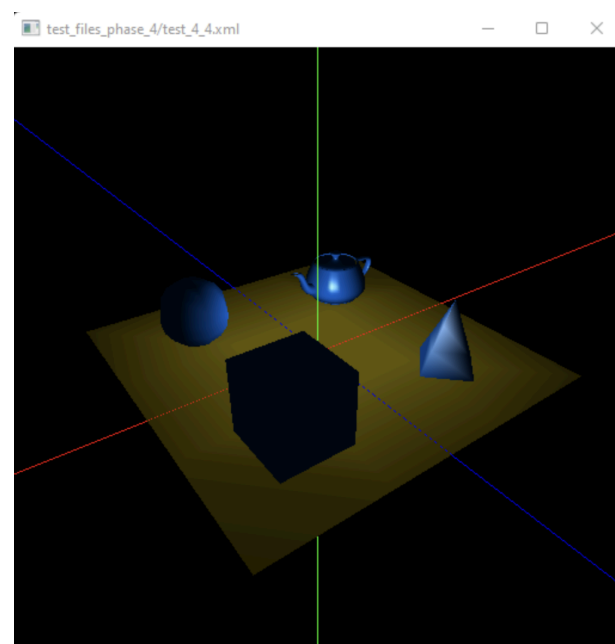
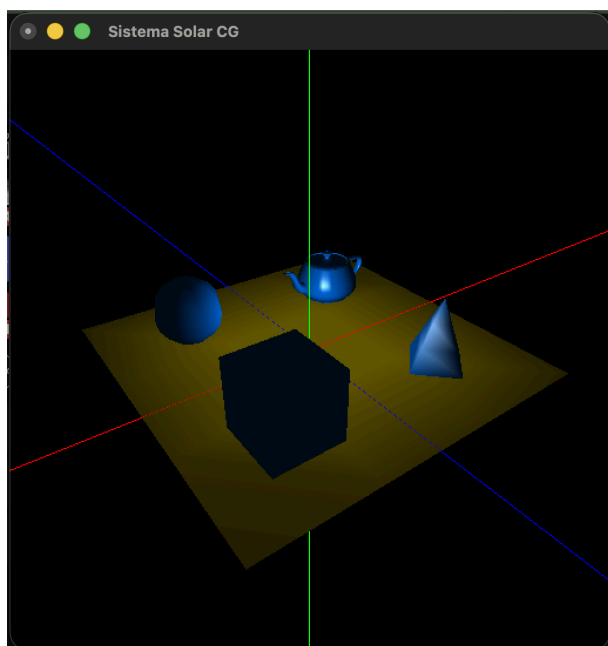
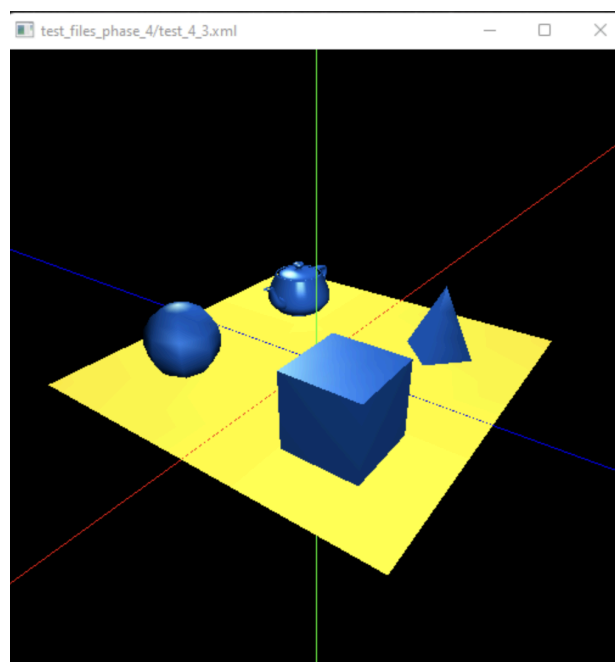
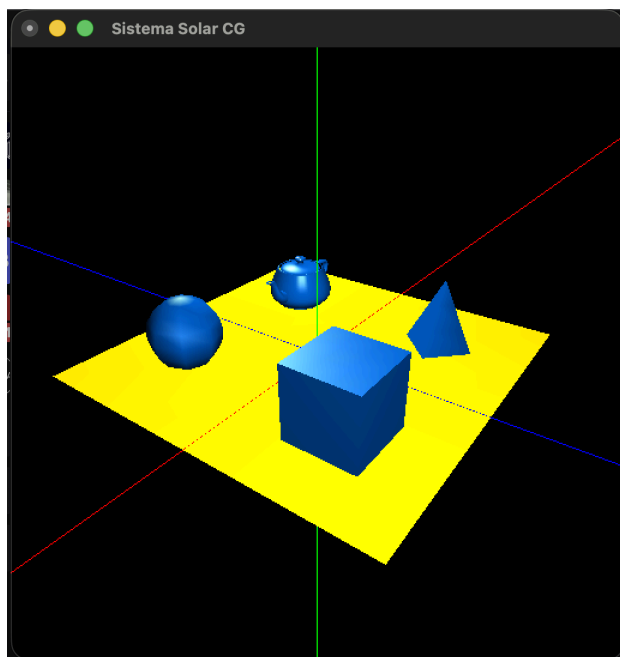
Figura 1: Vista panorâmica do Sistema Solar com texturização e iluminação completas.

8. Resultados e Testes

As cenas de teste fornecidas pela equipa docente para a Fase 4 cobrem progressivamente os diferentes requisitos visuais implementados, permitindo uma validação isolada e rigorosa de cada componente técnica do motor:

- **test_4_1 a test_4_3:** Validação isolada dos três tipos de fontes de luz (pontual, direcional e *spotlight*) com incidência em primitivas simples e materiais estáticos.
- **test_4_4 e test_4_5:** Validação do sistema de materiais, demonstrando o comportamento e a refração da luz face a diferentes combinações de componentes (diffuse, ambient, specular, emissive) e índices de reflexão (shininess).
- **test_4_6:** Validação do mapeamento de texturas (*UV mapping*) nas quatro primitivas base geradas pelo motor matemático (esfera, caixa, cone e plano).





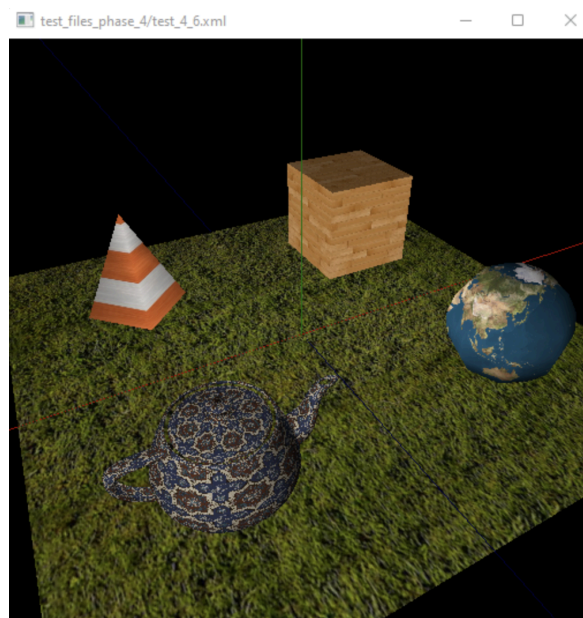
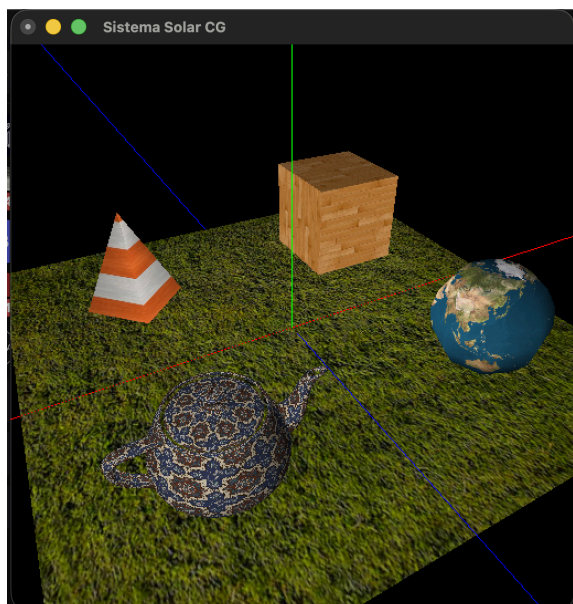
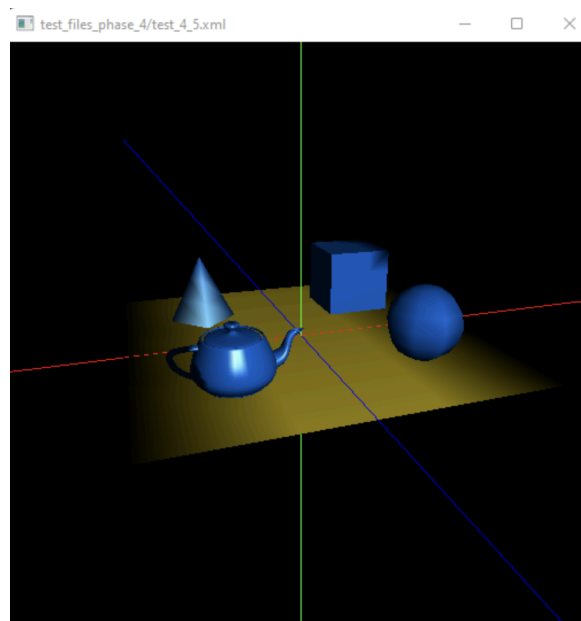
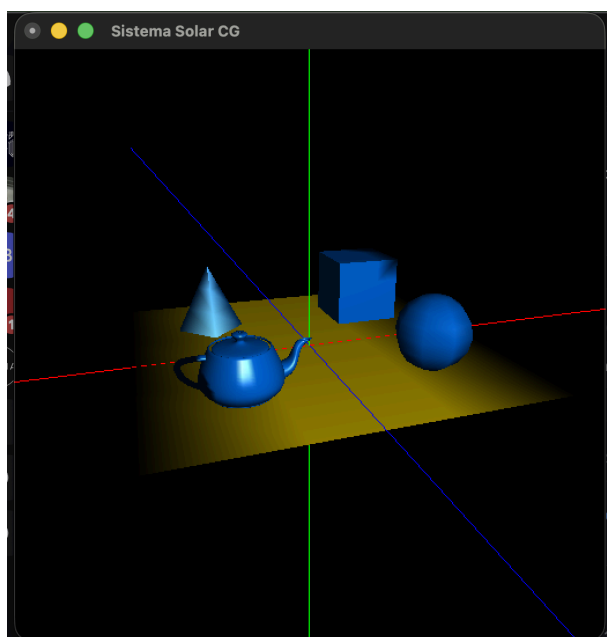


Figura 2: Validação das seis cenas de teste da Fase 4 (renderização do motor, esquerda, e as imagens de referência, direita).

9. Conclusão

A Fase 4 assinala a conclusão do desenvolvimento do motor gráfico. Em retrospectiva, as alterações finais foram deliberadamente cirúrgicas e localizadas: expansão do formato .3d matemático, alocação de três VBOs distintos por modelo, integração da biblioteca externa DevIL para processamento de texturas e a configuração rigorosa do *pipeline* de iluminação do OpenGL. Toda a robusta arquitetura herdada das fases anteriores, *Scene Graph* hierárquico, gestão da *Matrix Stack*, animações temporais com splines de Catmull-Rom e câmara orbital interativa, funcionou em pleno, validando as decisões de *design* tomadas no início do projeto.

O resultado final é um motor gráfico modular construído em C++, capaz de renderizar cenas hierárquicas, dinâmicas e complexas (evidenciado pela simulação do Sistema Solar) com iluminação precisa e texturas fotorealistas de alta fidelidade. Como trabalho futuro e evolução natural desta tecnologia, perspetiva-se a transição do *Fixed-Function Pipeline* clássico para a utilização de *Shaders* programáveis através de GLSL (*OpenGL Shading Language*), bem como a implementação de algoritmos de otimização espacial, como o *View Frustum Culling*, para maximizar o desempenho em cenas massivas de milhões de triângulos.

10. Referências Bibliográficas

1. OpenGL Registry: "OpenGL 2.1 Reference — glMaterial, glLight, glLightModel." <https://www.opengl.org/>
2. DevIL (Developer's Image Library): <http://openil.sourceforge.net>
3. LearnOpenGL.com: "Lighting" e "Textures." <https://learnopengl.com>
4. Shreiner, D. et al.: "OpenGL Programming Guide, 8th Edition" (The Red Book).
5. Catmull, E., & Rom, R.: "A class of local interpolating splines." (mantida das fases anteriores)
6. TinyXML-2: <https://github.com/leethomason/tinyxml2> (mantida das fases anteriores)